

Sparse Cholesky Factorization

MU5MAM30 - Implementation of the Finite Element Method

Anja Matic

1 Introduction

Cholesky decomposition is a direct method for solving symmetric, positive definite linear systems. In the context of finite element methods, it is particularly suited for time-dependent partial differential equations, where at each step the same linear system is solved with different right-hand sides. However, as the Cholesky factor has to be stored, there is a memory trade-off, especially since this factor is not necessarily sparse. In fact, this sparsity structure will heavily depend on the ordering of the vertices in the mesh.

In this project, a sparse Cholesky solver will be implemented and used in a solver for the Navier-Stokes equation. In particular, the importance of mesh ordering will be explored. The code is available at https://github.com/didiersmets/MU5MAM30_2025/tree/Matic. The source code and scripts related to the project can be found in the `projects` folder, in the sub-folders `include`, `source` and `utils`. Further information on how to build and run the project can be found in the `README` file in the `projects` folder.

2 Theoretical background

Let A be a symmetric positive definite (SPD) matrix. Denote the row index of the first sub-diagonal non-zero entry of A in column j as $parent(j)$ with the convention that $parent(j) = 0$ if no such entry exists. That is,

$$parent(j) = \min\{i : i > j, \text{ and } A_{i,j} \neq 0\}.$$

For $n \geq 1$ we define the n^{th} ancestor of j recursively by $parent^n(j) = parent^{n-1}(j)$ and $parent^0(j) = j$.

An elimination tree of a SPD matrix A , denoted $\mathcal{T}(A)$ or simply $\mathcal{T}$ is a directed graph with vertices $\{1, 2, \dots, n\}$ and edges $\{(j, parent(j)) : j = 1, \dots, n\}$, directed from child to parent. In general $\mathcal{T}(A)$ does not need to be connected, but can be a collection of disjoint components.

An elimination tree can be used to characterize the sparsity pattern of the Cholesky factor L of A . The following result links the sparsity structure of A to that of L :

Theorem 2.1. $(i, j) \in \mathcal{S}(L + L^t)$ if and only if there exists a path

$$p_1 = i \rightarrow p_2 \rightarrow \dots \rightarrow p_l = j \quad (1)$$

in $\mathcal{G}(A)$ such that $p_k \leq \min(i, j) \quad \forall 1 < k < l$.

Using elimination trees, such a path can be found. Let $\text{anc}_{\mathcal{T}}(j)$ denote the ancestors of a vertex j , where an ancestor of j in this context is a vertex i such that $\text{parent}^k(i) = j$.

Theorem 2.2. Let $\mathcal{T}$ be an elimination tree associated to the Cholesky factor L of a SPD matrix A . If i and j are vertices of $\mathcal{T}$, with $i < j$, such that $a_{i,j} = 0$, then $l_{ij} \neq 0$ if and only if there exists $k < j$ such that $j \in \text{anc}_{\mathcal{T}}(j)$ and $a_{ik} \neq 0$.

Additionally, the parent of a given vertex can be found using the path (1):

Theorem 2.3. Vertex i is the parent of vertex j in $\mathcal{T}$ if and only if i is the lowest numbered vertex satisfying $j < i$ for which there is a path (1).

By stepwise extending the elimination trees given by its submatrices, the elimination tree of A can be constructed. Suppose that $\mathcal{T}(A_{1:i-1,1:i-1})$ is already constructed and initialize $\mathcal{T}(A_{1:i,1:i})$ as $\mathcal{T}(A_{1:i-1,1:i-1})$. If there are no non-zero entries left of the diagonal in the i^{th} row of A , no paths starting from i will connect any of the vertices in the tree and i is added as an isolated vertex. Otherwise, $\mathcal{T}(A_{1:i-1,1:i-1})$ is traversed starting from each non-zero entry in the i^{th} row of A , until the current root of the tree is found, which is an index j such that $\text{parent}(j) = 0$ or $\text{parent}(j) = i$. In the first case, the tree is re-rooted such that $\text{parent}(j) = 1$. The details of the algorithm can be found in the appendix (Algorithm 1).

By Theorem 2.2, the sparsity pattern of L can be found by traversing the elimination tree. The algorithm is detailed in the appendix (Algorithm 2). This result also provides an idea of how to order the vertices in a mesh, in order to obtain a maximally sparse Cholesky factor. If two components of the mesh are separated by another component with high indices, this prevents paths from forming, hence preventing fill-in in the Cholesky factor.

The factorization was performed using sparse left looking Cholesky factorization, see Algorithm 3 in the Appendix. This is a sparse variant of the analogous dense algorithm, which relies on the sparsity pattern computed using Algorithm 2. In addition, the symbolic computation makes it possible to reserve an appropriate amount of memory before computing the Cholesky factor.

The reference for this section is [2].

3 Results and discussion

The tests of the symbolic phase and the Cholesky solver were carried out on the P_1 mass matrix. As the sparsity pattern of the stiffness matrix is identical to

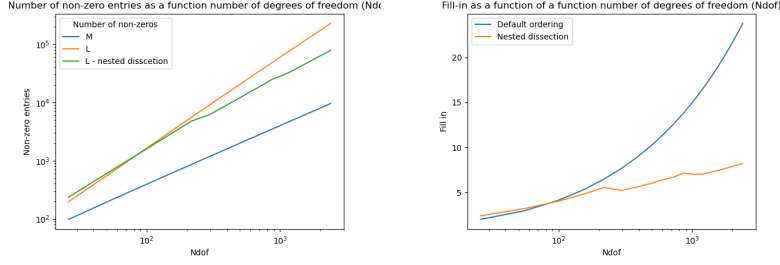


Figure 1: Number of non-zeros and fill-in of the mass matrix M associated to a mesh with either default ordering of its vertices or ordering given by nested dissection and its Cholesky factor L .

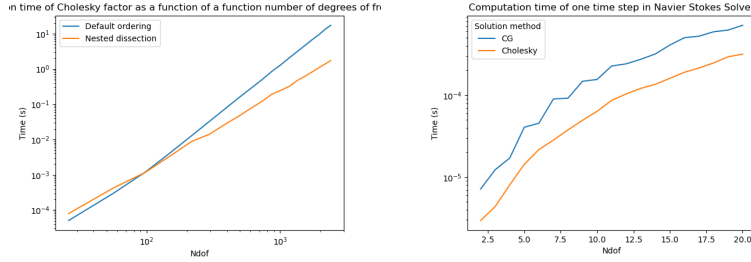


Figure 2: Time required to perform Cholesky factorization of FEM mass matrix and time required to perform one time-step in Navier Stokes simulation for different solution methods.

that of the mass matrix in this context, the results of Figure 1 applies to both of the P_1 matrices and further extends to the context of solving the complete linear system associated to the Navier-Stokes equation. The ordering of the vertices in the mesh has a considerable effect on the number of non-zeros in the Cholesky factor and therefore also on the fill-in. In a mesh with a 100 degrees of freedom, which corresponds to four subdivisions on each face of the cube, the effects are already present. Correct ordering is therefore a relevant consideration in practice.

Computing the Cholesky factor requires significantly less time for a mesh with a more optimal ordering, and the time required to perform one time step in the simulation is lower when using a Cholesky solver compared to the conjugate gradient method, as can be observed in Figure 2. The duration of one time step does not include the factorization step, however. For large matrices, the factorization step can take up to a minute if the sparsity structure of L is suboptimal. This further underlines the importance of vertex ordering. If the mesh is well ordered, the additional factorization cost, when amortized over a large number of time steps, may still result in better performance than the conjugate gradient method.

The conjugate gradient method requires $\mathcal{O}(\sqrt{\kappa} \log(\frac{1}{\epsilon}))$ iterations to converge, where κ denotes the condition number of the matrix and ϵ is the desired precision. Each iteration requires $\mathcal{O}(\text{nnz})$ operations, where nnz is the number of nonzero entries of the matrix. In contrast, solving the system $LL^T x = b$ via forward and backward substitution is linear in the number of nonzero entries of L and independent of the condition number. The $\mathbb{P}_1$ mass matrix is typically well conditioned, independently of mesh size [1]. The stiffness matrix, however, generally becomes ill conditioned as the mesh size decreases. This is only an issue for computing the stream term of the system, as the addition with the mass matrix will regularize the spectrum. To improve the performance of the conjugate gradient method, a preconditioner may be employed for the stiffness matrix. Finding a suitable preconditioner is nontrivial and introduces additional memory costs.

As Cholesky factorization is a direct method, it will have a higher precision than an iterative method such as the conjugate gradient method. In this aspect, the condition number plays a role also for the Cholesky method as it affects numerical stability.

In the context of solving the Navier-Stokes equations using $\mathbb{P}_1$ finite elements, if the mesh vertices can be ordered such that sufficient sparsity is preserved in the Cholesky factor, Cholesky factorization can be a solution method better adapted to the problem. But in certain cases, for example for a mesh that is adapted or refined as the simulation progresses or for very large problems, the conjugate gradient method might still be preferred.

4 Conclusion

In this project we have implemented a sparse Cholesky solver using elimination trees. We have shown that this method outperforms the conjugate gradient method when solving the linear system associated to the P_1 discretization of the Navier-Stokes equation, provided a well ordered mesh.

References

- [1] Pascal Frey. Chapter 7: The finite element approximation. MA691 Lecture Notes, Laboratoire J.-L. Lions, Université Paris-Sud / Universidad de Chile, 2008. PDF file, accessed on [your access date].
- [2] Jennifer Scott and Miroslav Tůma. *Algorithms for Sparse Linear Systems*. 01 2023.

A Appendix

Below follows a description of the algorithms implemented as a part of the project.

Algorithm 1 Construction of an elimination tree using path compression

Input: A , $n \times n$ SPD matrix and its undirected graph $\mathcal{G}(A)$

Output: Elimination tree $\mathcal{T}$ described by vector $parent$

```
for  $i = 1, \dots, n$  do
   $parent(i) = 0$ ,  $ancestor(i) = 0$ 
  for  $j \in adj_{\mathcal{G}}(i)$  and  $j < i$  do  $jroot = j$ 
    while  $parent(jroot) \neq 0$  and  $ancestor(jroot) \neq i$  do
       $l = ancestor(jroot)$ 
       $ancestor(jroot) = i$ 
       $jroot = l$ 
    end while
    if  $ancestor(jroot) = 0$  then
       $ancestor(jroot) = i$ 
       $parent(jroot) = i$ 
    end if
  end for
end for
```

Algorithm 2 Computation of row sparsity pattern of the Cholesky factor L
Input: A , $n \times n$ SPD matrix, its undirected graph $\mathcal{G}$, elimination tree $\mathcal{T}$ described by vector $parent$.
Output: Row sparsity pattern of the Cholesky factor L of A

```

for  $i = 1, \dots, n$  do
   $row_L(i) = \emptyset$ 
   $mark(i) = i$ 
  for  $k \in adj_{\mathcal{G}}(i)$  and  $k < i$  do
     $j = k$ 
    while  $mark(j) \neq i$  do
       $mark(j) = i$ 
       $row_L(i) = row_L(i) \cup \{j\}$ 
       $j = parent(j)$ 
    end while
  end for
end for

```

Algorithm 3 Simplified sparse left-looking Cholesky factorization
Input: A , $n \times n$ SPD matrix, $\mathcal{S}(L)$ pre-computed sparsity pattern of Cholesky factor
Output: Cholesky factor L .

```

 $l_{ij} = a_{ij}$  for all  $(i, j) \in \mathcal{S}(L)$ 
for  $j = 1, \dots, n$  do
  for  $k \in \{k < j \mid l_{jk} \neq 0\}$  do
    for  $i \in \{i \geq j \mid l_{ik} \neq 0\}$  do
       $l_{ij} = l_{ij} - l_{ik}l_{jk}$ 
    end for
  end for  $l_{jj} = \sqrt{l_{jj}}$ 
  for  $i \in \{i > j \mid l_{ij} \neq 0\}$  do  $l_{ij} = \frac{l_{ij}}{l_{jj}}$ 
  end for
end for

```
